



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**



SIGMA POWER
Power and Power Quality

WARAQ

Rapport Technique — Pipeline RAG

Module d'analyse métier des dos-
siers d'appels d'offres (rag_engine)

Réalisé par : Achraf AIT
LAHCEN

Date : 9 août 2026

Table des matières

1. Objectif du document	1
2. Contexte : de la classification à l'analyse métier	1
3. Comparaison des deux architectures	1
3.1. Schéma initial (v1)	1
3.2. Schéma modifié / Implémenté (v2)	2
4. Vue d'ensemble du flux d'exécution	2
5. Détail des composants	3
5.1. <code>rag_trigger.py</code> — Déclenchement asynchrone	3
5.2. <code>rag_document_extractor.py</code> — Extraction indépendante & détection de documents internes	3
5.2.1. Extraction multi-format & heuristique OCR	3
5.2.2. Détection & découpe de documents modèles internes	4
5.2.3. Orchestration de l'extraction (<code>traiter_extraction_rag()</code>)	4
5.3. <code>rag_service.py</code> — Orchestrateur du pipeline (<code>RAGPipelineService</code>)	5
5.3.1. Séquence d'exécution séquentielle du pipeline	5
5.3.2. Analyse critique & Point d'attention métier sur la portée contextuelle	6
5.4. <code>chunker.py</code> — <code>DocumentChunker</code>	6
5.4.1. Stratégie de segmentation par mots	6
5.4.2. Paramétrage et fonctionnement	6
5.5. <code>embeddings.py</code> — <code>OllamaEmbeddingService</code>	7
5.5.1. Choix du modèle et gestion des ressources	7
5.5.2. Point de vigilance sur la gestion des erreurs	7
5.6. <code>vector_store.py</code> — <code>ChromaDBManager</code>	7
5.6.1. Organisation par collection (Niveau Tender)	7
5.6.2. Idempotence et stockage	8
5.7. <code>prompts.py</code> — Prompts métier & ingénierie d'instruction	8
5.7.1. Analyse du prompt <code>GLM_OCR_PROMPT</code>	8
5.7.2. Analyse du prompt <code>GRANITE_EXTRACTION_PROMPT</code>	8
5.8. <code>schemas.py</code> & <code>models.py</code> — Structuration, validation et persistance	9
5.8.1. Validation applicative (<code>schemas.py</code>)	9
5.8.2. Persistance relationnelle (<code>models.py</code>)	9
6. Organisation du stockage disque	10
6.1. Analyse du découplage et rôle des répertoires	10
7. Points forts de l'architecture	11
8. Synthèse des points de vigilance techniques	12
9. Pistes d'amélioration	12
10. Conclusion	13

1. Objectif du document

Ce rapport décrit et explique en détail le **pipeline RAG** (*Retrieval-Augmented Generation*) intégré à Waraq, qui vient compléter la classification automatique (Qwen) déjà en place. Il couvre :

- L'évolution de l'architecture entre le schéma initial et le schéma modifié ;
- Le rôle précis de chaque fichier du module `rag_engine` ;
- L'organisation du stockage disque associée ;
- Les points forts de la conception ;
- Quelques incohérences techniques relevées dans le code actuel, utiles à corriger ou à clarifier avant la mise en production.

2. Contexte : de la classification à l'analyse métier

La classification (Qwen + PyMuPDF + RapidOCR) répond à une question simple : « **quel type de document est-ce ?** » (*RC, CPS, BDP, Acte d'Engagement...*).

Le pipeline RAG répond à une question plus riche : « **qu'est-ce que ce document dit, concrètement, et comment l'exploiter pour préparer une soumission ?** » — objet du marché, dates limites, pièces à fournir, critères d'évaluation, pénalités, garanties, etc.

C'est un changement de nature, pas seulement d'échelle : on passe d'une tâche de **catégorisation** à une tâche d'**extraction d'information structurée**, ce qui justifie un pipeline dédié, découplé de la chaîne de classification.

3. Comparaison des deux architectures

3.1. Schéma initial (v1)

Dans cette première version, le RAG **réutilise** le champ `extracted_text` déjà produit pendant la classification (PyMuPDF + RapidOCR), sans ré-extraction. C'est d'ailleurs ce que décrit ton paragraphe descriptif : « **Ce pipeline exploite le texte intégral déjà extrait par RapidOCR ONNX et stocké dans le champ `extracted_text`, sans nécessiter une nouvelle extraction OCR** »

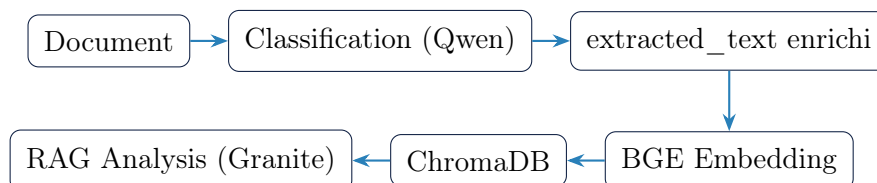
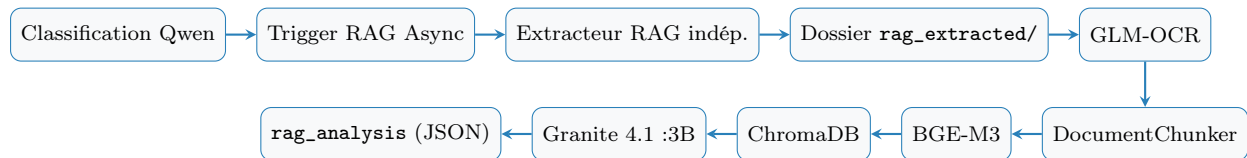


FIGURE 1 – Schéma du flux de données complet du pipeline Waraq

3.2. Schéma modifié / Implémenté (v2)

Ici, le RAG **ne réutilise plus** `extracted_text`. À la place, `rag_document_extractor.py` ré-exécute sa **propre** extraction (PyMuPDF + RapidOCR + détection de qualité OCR), indépendante du `learning_service` de classification.



/! Point d'attention / Clarification

Le paragraphe descriptif narratif initial correspond à l'architecture v1 (réutilisation de `extracted_text`), alors que le code implémente l'architecture v2 (extraction indépendante). Il est recommandé d'aligner le texte narratif du rapport sur le code réellement en production.

Pourquoi ce changement d'architecture a du sens :

- **Précision** : La classification cherche un signal *rapide* ; le RAG a besoin d'un texte *complet et fidèle*, y compris pour les tableaux.
- **Isolation** : Découpler les pipelines évite qu'une modification RAG n'impacte la classification.
- **Sécurité** : L'exécution en thread asynchrone séparé avec sa propre session SQLAlchemy (`rag_trigger.py`) garantit une étanchéité mémoire et SGBD.

4. Vue d'ensemble du flux d'exécution

1. Un document est classifié (Qwen) comme CPS, RC ou BDP.
2. `rag_trigger.lancer_rag_tender_async(tender_id)` déclenche un thread daemon dédié.
3. Une requête SQL sélectionne tous les documents qualifiés du tender (CPS, RC, BDP).
4. `rag_pipeline_service.execute_rag_pipeline()` s'exécute séquentiellement pour chaque document.
5. `traiter_extraction_rag()` ré-extrait le texte intégral du fichier source (PDF natif, OCR page-par-page si scanné, DOCX, Excel...).
6. Détection et extraction des **modèles internes** (CV, Acte d'Engagement...) en PDF séparés dans `rag_extracted/`.
7. Enrichissement du texte via **GLM-OCR** (nettoyage, structuration).
8. Découpage en chunks via `DocumentChunker`.
9. Vectorisation (BGE-M3) et indexation dans **ChromaDB** (collection unique par tender).
10. Recherche sémantique fixe ($\text{top-}k = 6$) sur l'ensemble de la collection.
11. Génération du JSON structuré (`rag_analysis`) par **Granite 4.1 :3B**.
12. Persistance du résultat, du statut et des métriques dans `RAGAnalysisResult`.

5. Détail des composants

5.1. `rag_trigger.py` — Déclenchement asynchrone

Rôle : Lancer le pipeline RAG **sans bloquer** le flux de classification.

- `threading.Thread(daemon=True)` : pas de blocage lors de l'arrêt de l'application.
- Ouverture d'une nouvelle session `SessionLocal()` dédiée pour éviter les conflits de concurrence SQLAlchemy.
- Filtrage stricte sur `is_classified == True` et `file_type` in `["CPS", "RC", "BDP"]`.
- Chaque document est traité **séquentiellement** dans une boucle `for`, avec gestion d'erreur individuelle (`try/except` par document) pour qu'un échec sur un document n'interrompe pas les autres.

5.2. `rag_document_extractor.py` — Extraction indépendante & détection de documents internes

Considéré comme le cœur de la nouvelle architecture d'ingestion, le module `rag_document_extractor.py` assure trois responsabilités distinctes et complémentaires : l'extraction multi-format, la détection/découpe fine de documents modèles internes, et l'orchestration globale de l'extraction.

5.2.1. Extraction multi-format & heuristique OCR

Le système prend en charge une large palette de formats de fichiers via des stratégies d'extraction adaptées (Tableau 1).

TABLE 1 – Stratégies d'extraction multi-format

Format	Méthode d'extraction
PDF (texte natif)	<code>page.get_text("text", sort=True)</code> via PyMuPDF (<code>fitz</code>).
PDF scanné	Fallback OCR page complète via RapidOCR (<code>extraire_page_complete_fast_ocr</code>).
DOCX	<code>python-docx</code> , extraction des paragraphes et tableaux linéarisés (comme séparateur).
DOC	Conversion préalable en PDF via LibreOffice headless, puis traitement PDF standard.
XLS / XLSX / XLSM	<code>pandas.read_excel</code> , lecture de toutes les feuilles et suppression des lignes vides.
Images (PNG, JPG, TIFF)	OCR direct via RapidOCR.

Pour les documents PDF, la détection d'une page scannée repose sur l'heuristique empirique `_ocr_est_de_mauvaise_qualite()`. Celle-ci déclenche le passage en OCR si le texte extrait est trop court (< 250 caractères), si le ratio de lettres alphabétiques est anormalement bas, ou si le texte présente une surreprésentation de caractères « parasites »

(+*#=<>\\|~). Cette méthode pragmatique nécessite toutefois une vigilance particulière sur les documents à forte densité de tableaux numériques.

5.2.2. Détection & découpe de documents modèles internes

Le dictionnaire `DOCUMENT_PATTERNS` associe à chaque typologie de document (CPS, RC, BDP, BPU, DPGF, Acte d'engagement, Déclaration sur l'honneur, Déclaration d'identité, CV, Déclarations fiscale et sociale) un jeu d'expressions régulières (*regex*) appliquées sur le texte préalablement normalisé (minuscules, apostrophes uniformisées, espaces compactés).

La fonction `extraire_pages_modeles_pdf()` parcourt les documents PDF (souvent de volumineux CPS ou RC) **page par page** afin d'identifier les annexes encadrées (modèles vierges de CV, déclarations, etc.). Une marge de contexte (`contexte_pages=1`) est conservée pour garantir la continuité des modèles s'étendant sur plusieurs pages. Chaque section identifiée est automatiquement extraite et reconstruite sous forme de sous-PDF indépendant (`fitz.open()` et `insert_pdf()`).

Ce traitement explique l'organisation de l'arborescence générée dans `rag_extracted/.../A0-123_.../`, où des sous-dossiers spécialisés (`CV/`, `DECLARATION_HONNEUR/`, `ACTE_ENGAGEMENT/`) sont créés aux côtés du dossier principal (`CPS/` ou `RC/`), un unique fichier source pouvant ainsi essaimer plusieurs documents sous-jacents.

5.2.3. Orchestration de l'extraction (`traiter_extraction_rag()`)

Le pipeline principal effectue les opérations suivantes :

1. **Construction de l'arborescence de sortie** : Structure normalisée du type `rag_extracted/AAAA/MM/JJ/<référence_nettoyée>/`.
2. **Extraction générique** : Exécution des routines d'extraction adaptées au format du fichier.
3. **Traitements spécifiques post-extraction** :
 - **Pour les PDF** : Identification et isolation des pages-modèles vers leurs sous-dossiers respectifs.
 - **Pour les fichiers Excel** : Si une feuille est identifiée comme un bordereau des prix ou décomposition (BDP, BPU, DPGF), elle est convertie en PDF via `ReportLab` (`extraire_bdp_excel_vers_pdf`) afin d'être directement exploitable par le workflow de génération documentaire.

Analyse de cohérence avec la conception théorique : Il convient de noter quelques écarts d'implémentation par rapport au schéma d'architecture cible :

- **Découpe fine limitée aux PDF et Excel** : Bien que le diagramme présente la détection des documents internes comme une étape générique post-extraction, la découpe fine page par page avec génération de sous-PDF n'est à ce jour effective que pour les flux PDF et Excel (`.xls/.xlsx/.xlsm`). Les fichiers DOC/DOCX bénéficient de l'extraction textuelle complète sans découpage interne automatique (ce qui reste acceptable dans la mesure où les fichiers Word multi-modèles sont rares dans le contexte des marchés publics).

- **Gestion des fichiers XML** : Le format `.xml` figurant au schéma conceptuel (« XML → conservation + lecture ») n'est pas encore pris en charge par `extraire_document_complet_pour_rag()` et bascule actuellement dans la branche des extensions non supportées.

5.3. `rag_service.py` — Orchestrateur du pipeline (`RAGPipelineService`)

Le service `RAGPipelineService` constitue le point d'entrée unique et l'orchestrateur central de l'ensemble du workflow RAG. Il expose la méthode `execute_rag_pipeline(db, document_id)`, exécutée de manière séquentielle pour chaque document éligible d'un appel d'offres.

5.3.1. Séquence d'exécution séquentielle du pipeline

L'exécution s'articule autour de douze étapes clés coordonnées rigoureusement :

1. **Initialisation & Changement d'état** : Récupération de l'entité `TenderDocument` et création (ou récupération) de la relation 1-à-1 avec `RAGAnalysisResult`. Le statut de traitement passe immédiatement à `"IN_PROGRESS"`.
2. **Construction de l'arborescence** : Définition du chemin cible `rag_extracted/AAAA/MM/JJ/<référence_nettoyée>/` en se basant sur `tender.extraction_date` et `tender.reference` (assainie des caractères spéciaux pour la compatibilité système).
3. **Extraction brute** : Appel de la fonction `traiter_extraction_rag()` pour obtenir le texte natif ou OCRisé indépendant.
4. **Enrichissement multimodal (GLM-OCR)** : Passage du document dans `_call_glm_ocr()` pour l'extraction avancée. En cas de succès, le texte enrichi remplace le texte brut ; en cas d'échec, un basculement (*fallback*) silencieux conserve le texte extrait à l'étape précédente.
5. **Persistance intermédiaire d'urgence** : Assignment `rag_entry.extracted_text = enhanced_text` suivie d'un *commit* dédié en base de données. Cette étape garantit la conservation permanente du texte extrait même en cas de défaillance ultérieure du pipeline.
6. **Segmentation (*Chunking*)** : Découpage du texte en segments optimisés via `DocumentChunker.create_chunks`.
7. **Vectorisation** : Génération des vecteurs d'embedding via `embedding_service.generate_embeddings`.
8. **Indexation vectorielle (ChromaDB)** : Stockage des vecteurs dans ChromaDB accompagnés de métadonnées enrichies (`tender_reference`, `document_id`, `file_name`, `doc_type`, `chunk_index`).
9. **Recherche sémantique globale** : Exécution d'une requête vectorielle orientée métier (« Objet, dates importantes, pièces administratives et techniques, clauses, critères d'évaluation et pénalités ») sur **l'ensemble de la collection** rattachée au dossier.
10. **Inférence LLM (Granite 4.1 :3B)** : Génération de l'analyse synthétique en fournissant au modèle Granite le contexte vectoriel extrait ainsi que le type de document (`doc.file_type`).

11. **Finalisation & Métriques** : Sauvegarde finale avec passage du statut à `COMPLETED`, enregistrement du JSON brut (`rag_analysis`), du nom de la collection Chroma, du nombre de `chunks`, et des métriques de performance (durées d'extraction, d'embedding, d'inférence LLM et durée totale).
12. **Gestion des exceptions** : Capture globale des erreurs avec basculement du statut vers `FAILED` et enregistrement du message d'erreur explicatif (`error_message`).

5.3.2. Analyse critique & Point d'attention métier sur la portée contextuelle

Note : Point d'attention métier sur la portée contextuelle

Au step 9, `query_tender_context()` est appelée **sans** le paramètre `document_id`, alors que la méthode le supporte. Cela signifie que pour un tender ayant un CPS, un RC et un BDP déjà indexés, **chaque document interroge le même contexte global** (l'ensemble du dossier), avec la **même requête générique**.

Résultat probable : les trois `RAGAnalysisResult` (un par document) convergent vers un JSON très similaire, alors que `doc.file_type` est transmis à Granite comme si l'analyse était spécifique au document.

Deux pistes d'évolution sont envisageables selon l'intention métier :

- **Pour une analyse consolidée par tender** : Générer un seul `rag_analysis` au niveau du dossier (Tender) au lieu de l'associer individuellement à chaque document ;
- **Pour une analyse spécifique par document** : Transmettre `document_id=str(doc.id)` à `query_tender_context()` pour restreindre la recherche vectorielle aux seuls chunks du document courant.

5.4. chunker.py — DocumentChunker

Le module `chunker.py` implémente la stratégie de découpage textuel du pipeline RAG via la classe `DocumentChunker`.

5.4.1. Stratégie de segmentation par mots

Le choix s'est porté sur un découpage **par mots** (s'appuyant sur `text.split()`) plutôt que par caractères ou tokens. Ce choix architectural est explicitement justifié par la nécessité de garantir la compatibilité bilingue Français/Arabe : la séparation sur les espaces reste valide quel que soit le script linguistique, contrairement à une découpage strict par caractères qui pose des problèmes de segmentation avec l'alphabet arabe.

5.4.2. Paramétrage et fonctionnement

- **Configuration** : `chunk_size` = 800 mots et `overlap` = 150 mots (configurés au niveau de l'initialisation de `RAGPipelineService`).
- **Fenêtre glissante** : Le pas de progression est calculé dynamiquement (`step` = `chunk_size - overlap`).
- **Métadonnées embarquées** : Chaque segment conserve son identifiant (`chunk_id`), sa taille (`word_count`) ainsi qu'un champ optionnel `page_number` (non renseigné à ce stade, le texte issu de l'enrichissement GLM-OCR ne conservant pas la pagination d'origine).

- **Contrôle de cohérence** : Une validation explicite `overlap < chunk_size` est exécutée dès l'instanciation.

Cette approche offre une implémentation simple, robuste et totalement dépourvue de dépendances lourdes (absence de tokenizer externe), en parfait accord avec les contraintes d'exécution *CPU-only* du projet.

5.5. `embeddings.py` — `OllamaEmbeddingService`

Le service `OllamaEmbeddingService` assure la vectorisation des segments de texte en interagissant directement avec l'API HTTP d'Ollama via le point d'entrée `/api/embed`.

5.5.1. Choix du modèle et gestion des ressources

Le modèle utilisé est configuré via `settings.MODEL_EMBEDDINGS` (modèle multilingue BAAI/bge-m3). Pour répondre à la contrainte d'exécuter l'ensemble de la chaîne sur des machines hôtes disposant de ressources limitées (notamment 16 Go de RAM), la requête HTTP injecte le paramètre `keep_alive = 0` (`OLLAMA_KEEP_ALIVE=0`). Cela force le déchargement immédiat du modèle de la mémoire RAM/VRAM dès la fin de l'inférence, libérant ainsi les ressources pour les étapes suivantes (notamment l'exécution des LLM).

5.5.2. Point de vigilance sur la gestion des erreurs

En cas de réponse HTTP différente du code 200 OK, le service capture l'erreur et retourne une liste vide (`[]`) au lieu de lever une exception.

Point d'attention technique

Ce comportement de basculement silencieux est à surveiller : si `generate_embeddings()` renvoie une liste vide alors que des chunks existent, l'appel ultérieur à `collection.upsert()` risque d'échouer ou de générer des incohérences en amont de l'indexation vectorielle.

5.6. `vector_store.py` — `ChromaDBManager`

La gestion du stockage vectoriel est confiée à la classe `ChromaDBManager`, qui s'appuie sur une instance locale de `PersistentClient` configurée avec `anonymized_telemetry = False` (adapté aux contraintes de confidentialité des déploiements sur site / *on-premise*).

5.6.1. Organisation par collection (Niveau Tender)

L'architecture retient une règle structurante : **une collection unique ChromaDB par appel d'offres** (`tender_<référence_nettoyée>`), et non une collection individuelle par document. Ce choix offre deux avantages majeurs :

- **Recherche sémantique unifiée** : Il permet de requêter simultanément l'ensemble des pièces d'un même dossier (CPS, RC, BDP) au cours de la même passe d'inférence RAG.
- **Gestion granulaire du cycle de vie** : La méthode `delete_document_chunks()` permet de supprimer sélectivement les seuls segments d'un document précis (via un filtre

`where={"document_id": ...}`) sans affecter les autres pièces du tender (très utile lors des opérations de re-scan ou de mise à jour).

5.6.2. Idempotence et stockage

La méthode `store_document_chunks()` exploite la fonction `upsert()` de ChromaDB avec une convention de nommage déterministe pour les identifiants de chunks :

$$ID = doc_ \{document_id\}_chunk_ \{i\}$$

Cette approche garantit l'idempotence des traitements : réexécuter le pipeline sur un document existant remplace proprement ses anciens vecteurs dans la collection au lieu de les dupliquer.

5.7. `prompts.py` — Prompts métier & ingénierie d'instruction

Le module `prompts.py` rassemble les gabarits d'instructions destinés aux modèles de langage du pipeline.

5.7.1. Analyse du prompt `GLM_OCR_PROMPT`

Ce prompt est conçu pour la mise en forme et le nettoyage préalable du texte (préservation des montants, dates, références, structure des tableaux, gestion du bilinguisme FR/AR, et consigne stricte d'anti-hallucination). Il intervient comme une étape de normalisation avant l'analyse fonctionnelle.

Points d'attention techniques sur l'exécution de GLM-OCR

- **Exploitation textuelle exclusive (Non-VLM) :** Bien que présenté sous la dénomination de modèle vision-langage (VLM), l'appel effectif dans `_call_glm_ocr()` n'injecte actuellement que du texte brut (`raw_text`). Le bloc de code chargeant l'image codée en base64 dans le *payload* (`payload["images"] = [...]`) est commenté. GLM-OCR opère donc actuellement comme un re-formateur de texte et non comme un processeur visuel direct, ce qui crée une nuance avec les capacités visuelles cibles du schéma d'architecture.
- **Redondance du contexte :** Le texte brut `raw_text` est injecté à deux reprises lors de la construction finale du prompt (une première fois dans le corps du template, puis une seconde fois par concaténation dans le *payload*). Bien que sans impact fonctionnel direct, cette duplication entraîne une consommation inutile de la fenêtre de contexte en tokens.

5.7.2. Analyse du prompt `GRANITE_EXTRACTION_PROMPT`

Ce prompt régit l'extraction métier structurée. Il impose des règles de validation rigoureuses :

- Obligation de retourner la valeur `null` pour toute donnée absente sans jamais inventer d'information.

- Distinctions métier explicites (ex. distinction stricte entre estimation financière et caution provisoire, ou entre délai d'exécution et durée de validité des offres).

Cette approche constitue une bonne pratique éprouvée pour les documents administratifs et juridiques où la précision prime sur la créativité.

5.8. `schemas.py` & `models.py` — Structuration, validation et persistance

Les modules `schemas.py` et `models.py` assurent respectivement la modélisation de l'échange de données (Pydantic) et la persistance relationnelle (SQLAlchemy).

5.8.1. Validation applicative (`schemas.py`)

Le schéma Pydantic `TenderRAGAnalysisResult` définit le contrat d'interface et la structure du JSON métier attendu (`ImportantDates`, `RequiredDocuments`, `clauses`, critères et pénalités).

Écart de nommage détecté

Une incohérence subsiste entre le prompt et le schéma Pydantic : `GRANITE_EXTRACTION_PROMPT` demande la clé JSON `"maitre_d_ouvrage"`, alors que le schéma Pydantic définit l'attribut `maitre_douvrage` (sans le second tiré bas). Dans l'éventualité où le schéma serait utilisé pour parser et valider automatiquement la sortie du LLM (actuellement stockée au format JSON brut), cette divergence provoquerait un échec de sérialisation. Un alignement de nommage est à prévoir.

5.8.2. Persistance relationnelle (`models.py`)

L'entité SQLAlchemy `RAGAnalysisResult` conserve le statut du traitement via l'énumération `RAGStatus` (`PENDING`, `INDEXING`, `ANALYZING`, `COMPLETED`, `FAILED`), le texte enrichi, le JSON d'analyse (`JSONB`) et les métriques de traçabilité (durées d'exécution, nombre de chunks, modèles utilisés).

Incohérences techniques et refactorisations requises

1. **Incompatibilité de statut dans l'énumération** : Dans `rag_service.py`, la valeur "IN_PROGRESS" est assignée au champ `rag_entry.status`. Or, cette valeur est absente de l'énumération `RAGStatus`. Selon la rigueur du pilote de base de données, cette affectation risque de déclencher une erreur d'intégrité à l'écriture. Il convient d'aligner le code en utilisant `RAGStatus.INDEXING` ou `RAGStatus.ANALYZING`, ou d'ajouter le statut `IN_PROGRESS` à l'enum.
2. **Redondance d'attribut SQLAlchemy** : Le champ `error_message` est déclaré à deux reprises dans la classe `RAGAnalysisResult`. La seconde déclaration écrase la première lors du cartographie ORM ; un nettoyage est nécessaire pour garantir la lisibilité du code et prévenir d'éventuels conflits lors des migrations Alembic.
3. **Gestion du chemin de stockage** : `BASE_STORAGE_DIR` est défini dans `rag_service.py` sous forme d'une chaîne de caractères brute (ex. `r"C:\...\data_storage"`) tout en étant manipulé avec l'opérateur de concaténation de chemins `/`, propre au module `pathlib.Path`. L'exécution directe de cette ligne lève une erreur de type (`TypeError`). Le chemin doit être explicitement instancié via `Path(BASE_STORAGE_DIR)` ou configuré dynamiquement à partir des variables d'environnement centralisées dans `config.py`.

6. Organisation du stockage disque

L'organisation physique du répertoire de données `data_storage/` répond à une contrainte stricte de découplage et d'isolement des responsabilités entre les différents sous-systèmes (scraping, classification et pipeline RAG).

```
data_storage/
├── classified/.../TYPE/fichier.pdf .. Sortie classification (Qwen)
├── extracted/.../ao_xx/*.pdf ..... Pièces brutes de l'archive
├── rag_extracted/AAAA/MM/JJ/<ref_A0>/ Sortie DÉDIÉE du pipeline
    RAG
    ├── CPS/
    ├── RC/
    ├── BDP/
    ├── CV/
    ├── DECLARATION_HONNEUR/
    ├── ACTE_ENGAGEMENT/
    ├── DECLARATION_IDENTITE/
├── chroma_db/ ..... Index vectoriel persistant (SQLite + HNSW)
```

FIGURE 2 – Structure de l'arborescence du répertoire `data_storage/`

6.1. Analyse du découplage et rôle des répertoires

Chaque sous-dossier joue un rôle distinct dans le cycle de vie de la donnée :

- **extracted/** : Reçoit le contenu brut issu de la décompression des archives d'appels d'offres scrapées (ao_xx/). Ce dossier constitue la source primaire inchangée.
- **classified/** : Stocke les résultats triés et catégorisés issus du pipeline de classification basé sur Qwen, organisés par typologie documentaire.
- **rag_extracted/** : Répertoire **exclusivement dédié** aux traitements du pipeline RAG. Il adopte un partitionnement temporel et métier (AAAA/MM/JJ/<référence_A0>/). C'est dans ce répertoire que s'effectue la création des sous-dossiers spécialisés (CPS/, RC/, CV/, ACTE_ENGAGEMENT/, etc.) hébergeant les sous-PDFs générés lors de la détection de modèles internes.
- **chroma_db/** : Héberge la base de données vectorielle persistante exploitant un moteur SQLite associé à des index HNSW (*Hierarchical Navigable Small World*) pour les recherches de similarité cosinus rapides.

Bénéfice architectural : Ce cloisonnement constitue une bonne pratique d'ingénierie logicielle : les répertoires **extracted/** (zone tampon brute), **classified/** (domaine classification) et **rag_extracted/** (domaine RAG) sont totalement étanches et ne risquent aucune collision d'écriture, même lorsqu'ils manipulent simultanément les mêmes fichiers sources. Cette étanchéité au niveau du système de fichiers vient consolider l'indépendance fonctionnelle déjà garantie dans la couche applicative.

7. Points forts de l'architecture

Le pipeline RAG conçu se distingue par plusieurs choix d'ingénierie qui garantissent sa robustesse, sa légèreté matérielle et sa pertinence métier :

- **Isolation totale du sous-système :** Le traitement RAG s'exécute dans des threads dédiés, avec des sessions de base de données distinctes et une arborescence de stockage propre (**rag_extracted/**). Cette séparation évite tout risque de régression sur la brique de classification déjà déployée en production.
- **Conception CPU-only cohérente et souveraine :** L'ensemble du pipeline (Rapi-dOCR basé sur le moteur ONNX, embeddings BGE-M3 et modèle de langage Granite 4.1:3B gérés via Ollama) est optimisé pour tourner intégralement sur architecture CPU local. La libération active de la mémoire via `keep_alive = 0` permet de maintenir l'empreinte RAM sous le seuil des 16 Go sans dépendance GPU.
- **Prise en charge native du multilinguisme (FR/AR) :** La gestion du bilinguisme français/arabe est intégrée à chaque niveau : choix d'un modèle d'embedding multilingue performant (**bge-m3**), segmentation textuelle par mots (`text.split()`) agnostique du sens de lecture, et prompts métier imposant le respect des nuances linguistiques.
- **Détection & extraction automatique des documents modèles :** L'isolation page par page des annexes encadrées (modèles vierges de CV, déclarations sur l'honneur, actes d'engagement) au sein des volumineux fichiers PDF (CPS/RC) évite tout traitement manuel des dossiers scannés massifs.
- **Idempotence de l'indexation vectorielle :** L'utilisation de la méthode `upsert()` avec des identifiants de chunks déterministes (`doc_{id}_chunk_{i}`) garantit qu'une réexécution du pipeline sur un document met à jour ses vecteurs sans jamais générer de doublons dans ChromaDB.

- **Consignes d’anti-hallucination explicites** : L’ingénierie d’instruction appliquée au modèle Granite impose le retour systématique de la valeur `null` en cas de donnée manquante, répondant aux exigences de rigueur juridique et administrative des marchés publics.
- **Traçabilité et télémétrie fine** : Chaque analyse enregistrée dans `RAGAnalysisResult` conserve la durée de chaque étape (extraction, embedding, LLM), le nombre de segments générés et l’identifiant de la collection vectorielle, fournissant une base solide pour un futur tableau de bord de performance.

8. Synthèse des points de vigilance techniques

L’analyse approfondie du code source a mis en évidence plusieurs écarts d’implémentation et points d’attention technique qu’il convient de corriger avant le passage en production.

Recommandation de refactorisation : La résolution prioritaire des points 1, 2 et 8 permettra de sécuriser la stabilité à chaud du pipeline, tandis que l’alignement des points 4 et 5 garantira la cohérence fonctionnelle des résultats d’analyse produits par le modèle Granite.

9. Pistes d’amélioration

Afin d’optimiser davantage la valeur ajoutée métier et la résilience du système, plusieurs axes d’évolution architecturale et logicielle sont préconisés :

1. **Consolidation de l’analyse au niveau Dossier (*Tender-level*)** : Repenser la granularité de l’analyse pour générer une synthèse RAG unique et consolidée par appel d’offres (regroupant l’analyse combinée du CPS, du RC et du BDP) plutôt qu’une analyse fragmentée par document. Cela éliminerait la redondance actuelle des rapports générés et offrirait une vue synthétique directe pour les utilisateurs métier.
2. **Résilience des communications HTTP (Moteur de *Retry & Backoff*)** : Intégrer un mécanisme de tentative automatique avec temporisation exponentielle (*exponential backoff*) sur l’ensemble des appels HTTP à destination de l’API Ollama (GLM-OCR, embeddings BGE-M3 et extraction Granite). Cette mesure permettra d’absorber les pics de charge RAM et les délais de chargement à chaud des modèles en mémoire.
3. **Validation stricte et découplage du schéma JSON** : Activer le parsing et la validation Pydantic systématique des réponses générées par Granite via le schéma `TenderRAGAnalysisResult` avant leur persistance en base de données. Cet ajustement (qui nécessite un alignement préalable de la clé `maitre_douvrage`) permettra d’intercepter immédiatement les dérives de structure ou de format du LLM.
4. **Arbitrage fonctionnel sur le rôle de GLM-OCR** : Activer le traitement réellement visuel (encodage image base64 + prompt visuel) ou formaliser le statut de GLM-OCR comme simple re-formateur textuel. Clarifier ce choix permettra de confirmer la valeur

ajoutée de cette étape intermédiaire par rapport à un passage direct du texte extrait au modèle Granite.

5. **Extension de la détection de modèles aux fichiers Word (.docx)** : Adapter la logique de découpe fine page par page (`extraire_pages_modeles_pdf`) aux documents Word afin d'isoler automatiquement les annexes et formulaires vierges encastres dans des fichiers DOCX combinés multi-modèles.
6. **Prise en charge du format .xml** : Étendre le chargeur de documents `rag_document_extractor.py` pour lire et traiter les fichiers XML structurés susceptibles d'être présents dans les archives d'appels d'offres scrapées.

10. Conclusion

Le pipeline RAG de la plateforme Waraq introduit une **couche d'analyse métier autonome**, performante et parfaitement dimensionnée pour des environnements d'exécution contraints en ressources (*CPU-only*). Il répond efficacement aux spécificités des marchés publics grâce à une chaîne de traitement maîtrisée : extraction multi-format adaptée au bilinguisme français/arabe, détection fine de sous-documents annexes, indexation vectorielle unifiée au niveau du dossier et extraction structurée JSON sous contrainte stricte d'anti-hallucination portée par Granite 4.1 :3B.

La décision d'isoler totalement ce pipeline du module de classification (exécution dans un thread asynchrone, sessions de base de données dédiées et arborescence de stockage étanche `rag_extracted/`) constitue **le choix d'architecture le plus solide et le plus structurant du système**. Ce découplage garantit la stabilité de la brique de classification déjà en production tout en offrant l'agilité nécessaire aux évolutions futures de la couche RAG.

Les ajustements techniques identifiés lors de cette analyse (alignement des types de chemins, mise en conformité de l'énumération des statuts, filtrage contextuel par document et activation du parsing Pydantic) représentent des corrections classiques d'ingénierie applicative. Leur prise en compte préalable à la mise en production permettra d'éliminer les comportements silencieux indésirables et de maximiser la pertinence des résultats d'analyse restitués aux utilisateurs.

TABLE 2 – Matrice des anomalies et points de vigilance techniques

#	Fichier	Problème identifié	Impact & Risque opérationnel
1	rag_service.py	BASE_STORAGE_DIR est instancié comme une <code>str</code> mais manipulé avec l'opérateur / propre à <code>Path</code> .	Erreur d'exécution probable (<code>TypeError</code>) lors de la concaténation de chemins.
2	rag_service.py	Le statut "IN_PROGRESS" est assigné à <code>rag_entry.status</code> mais n'existe pas dans l'enum <code>RAGStatus</code> .	Risque d'erreur d'intégrité BDD à l'écriture selon la rigueur du driver SQLAlchemy.
3	models.py	Le champ <code>error_message</code> est déclaré à deux reprises dans la classe <code>RAGAnalysisResult</code> .	Écrasement silencieux au niveau du mapper ORM; dette technique pour les migrations.
4	schemas.py / prompts.py	Clé JSON "maitre_d_ouvrage" dans le prompt vs <code>maitre_douvrage</code> dans le schéma Pydantic.	Échec de sérialisation si le schéma est un jour appliqué pour valider la sortie LLM.
5	rag_service.py	Recherche sémantique globale non restreinte par <code>document_id</code> au step 9 du pipeline.	Génération d'analyses JSON quasi-identiques pour toutes les pièces d'un même tender.
6	rag_service.py	Encodage image base64 commenté dans <code>_call_glm_ocr</code> (seul le texte brut est transmis).	GLM-OCR fonctionne en simple nettoyeur de texte sans exploiter l'analyse visuelle VLM.
7	rag_document_extractor.py	Absence de branche de traitement pour le format <code>.xml</code> dans l'extracteur.	Rejet systématique des fichiers XML dans la branche « extension non supportée ».
8	embeddings.py	Renvoi d'une liste vide [] en cas d'échec HTTP de l'API Ollama au lieu d'une exception.	Masquage de l'erreur entraînant un plantage ultérieur sur <code>collection.upsert()</code> .